



Presenter 2 — Architecture, flows & cryptography

Slides 8 – 11 · target time ≈ 8 minutes

Your name: _____ Roll no.: _____

You take over from: Presenter 1 (research gap) · You hand over to: Presenter 3 (sandbox & policy, live demo)

How to use this script

The **Say** text is written to be spoken at a natural pace (about 130 words a minute). You do not need to read it word for word: learn the first and last sentence of each slide and the numbers, and speak the middle in your own words. *Italic cues* tell you what to point at. Everything technical here matches the code in the repository.

The whole talk at a glance

Part	Presenter	Slides	Time	What it covers
1	Presenter 1	1 – 7	≈ 8 min	Title, abstract, introduction, problem & objectives, literature review (2 slides), research gap
2	Presenter 2	8 – 11	≈ 8 min	System architecture, install flow, attestation & logging flow, cryptography & the Warp log
3	Presenter 3	12 + live demo A	≈ 9 min	Heddle sandbox & Weave policy engine; live scenarios 1, 2, 3, 4, 7
4	Presenter 4	live demo B + 13 – 16	≈ 9 min	Live scenarios 5, 6, 8 and the full-run console; results, demo recap, conclusion, references, Q&A;

Slide 8 · System architecture (2 min)

On screen: trace AUR → rebuilders → Warp → witnesses, then the long indigo arrow back into the client.

Say

Here is the whole system. The user talks only to the **loom client**, the dashed box, which is our trusted computing base. Inside it: **Weave**, the policy engine that makes decisions; **Heddle**, the build sandbox; the **Warp client**, which verifies the log; **Shuttle**, which fetches artifacts by hash; and the AUR backend, the only part that knows anything about the AUR.

Outside are independent parties. The **AUR** is read-only to us: we read recipes, and we never execute a recipe to learn its metadata; we parse the .SRCINFO file instead. Three **Thread rebuilders**, run by different organisations with different toolchains, rebuild every package and sign what they built. Those signed attestations go into the **Warp log**, and three **witnesses** cosign the log's checkpoints. **Shuttle peers** serve the built artifacts by content address.

At the bottom, pacman performs the final install and the Linux kernel provides the sandbox primitives. The key point is the note on the right: every arrow into the client carries data that is verified by a hash or a signature before it is used. If any verification fails, Loom refuses. It never falls back to trusting the data.



Terms to be ready to explain: trusted computing base, content address, attestation, checkpoint, cosignature.

Slide 9 · Install flow: decide before anything runs (2 min)

On screen: follow the numbers: top row left to right, the diamond, then the bottom row right to left.

Say

This is what happens when you type **loom install**. Step one resolves the package and its dependencies, dependencies first. Step two fetches the recipe and parses `.SRCINFO`, and no package code runs. Step three gathers evidence: it verifies the log, collects the attestations, and checks continuity, quarantine and security advisories.

Step four is the Weave decision. If anything fails, Loom refuses right there and prints the rule that fired, the evidence and the exact command to proceed if the user accepts the risk. **At that point no package code has executed.**

If it is allowed, step five obtains the artifact. Loom prefers the **attested artifact**, fetched from peers by its SHA-256 hash. So when k independent rebuilders agree, the `PKGBUILD` never runs on the user's machine at all. Only if no peer can serve it does Loom build locally, inside Heddle.

Then Loom inspects what the package would install, strips its install script so pacman can never run it as root, and re-evaluates, because the files themselves can still cause a block, for example a pacman hook. Finally it installs, runs any install script inside the sandbox, and records a baseline for future continuity checks.

Terms to be ready to explain: FR-3.9 (never execute a build script outside confinement), content addressing, `.INSTALL`.

Slide 10 · Attestation and logging flow (sequence) (1 min 45 s)

On screen: move down the numbered arrows 1 to 10.

Say

This sequence diagram shows where the evidence comes from; time runs downward. A rebuilder fetches the recipe with its pinned sources and **builds it twice** with deliberately different time zones, locales, umasks and build paths. That is the reprotest technique from the Reproducible Builds project. If both SHA-256 digests match, the build is reproducible.

The rebuilder then signs an in-toto statement, wrapped in a DSSE envelope, with its own Ed25519 key. The statement binds the artifact hash to the source commit and to the rebuilder's identity and toolchain. It submits this to Warp.

Warp checks that the signer is an admitted rebuilder, appends the record as a new leaf, signs a new checkpoint and sends it to each witness together with a **consistency proof**. A witness verifies that proof against the last checkpoint it cosigned and only then returns a cosignature.

Finally the client fetches the checkpoint, requires at least two of the three cosignatures, checks consistency with the head it saved last time, and verifies inclusion of every attestation it uses.

Terms to be ready to explain: reprotest, in-toto, DSSE, consistency proof, inclusion proof.



Slide 11 · Cryptography and the Warp transparency log (2 min)

On screen: point at the code box when you say 0x00 and 0x01; trace d2 → h1.1 → h2.0 → root, then the three amber boxes.

Say

We invent no cryptography; that is a hard constraint in our requirements. Everything uses two audited Rust libraries: ed25519-dalek and sha2. **SHA-256** gives every artifact and record a content address. **Ed25519** signs attestations, checkpoints and cosignatures, and we verify with `verify_strict`, which rejects malleable and small-order signatures.

Two subtler choices matter. **DSSE's pre-authentication encoding** signs the payload type and the body, each prefixed with its length, so nothing can be spliced or reinterpreted; and because the in-toto *predicate type* is inside the signed body, a signature on an attestation can never be reused as a signature on a revocation. And **canonical JSON**, with sorted keys and no floating-point numbers, means the same record always produces the same bytes, so anyone can re-verify a signature.

On the right is the Merkle tree. Leaves are hashed with a 0x00 prefix and interior nodes with 0x01, as in RFC 6962; that domain separation stops an attacker from passing off an interior node as a leaf. To prove that record d2 is in the log, the server sends only the three amber sibling hashes: log base two of eight. With a million records that is only twenty hashes.

Why witnesses? A dishonest log could show the victim one history and everyone else another: a **split view**. A witness only cosigns after checking consistency with what it cosigned before, so an honest witness vouches for one linear history. With two of three witnesses required, any two quorums share at least one witness, so a split view is **prevented** while at most one witness colludes, and detected beyond that.

Terms to be ready to explain: Ed25519, SHA-256, DSSE PAE, canonical JSON, RFC 6962, inclusion and consistency proofs, quorum intersection $2t - n$.

Handover line

“So the evidence is signed, logged and witnessed. What stops a malicious build in the first place, and how does Loom turn all this evidence into a decision? [Presenter 3] will explain Heddle and Weave, and then show you all of this running live.”



Questions the jury is likely to ask you

Answer in two or three sentences, then stop. If a question belongs to another presenter's area, say "My teammate covered that, [name] can answer" and hand it over.

Q. Why a Merkle tree and not a blockchain?

We need append-only history with cheap proofs, not consensus among strangers. A Merkle log gives $O(\log n)$ inclusion and consistency proofs, and a few independent witnesses prevent equivocation cheaply. Parakeet [13] and OPTIKS [14] show this works in production. Our requirements also rule out blockchain and tokens (CON-2).

Q. What exactly does a witness check before cosigning?

It receives the new checkpoint, the size it last cosigned, and a consistency proof between the two. It verifies the log's signature and the proof; if the new tree does not extend the old one it refuses with a conflict. Otherwise it signs a cosignature/v1 message that includes a timestamp. The witness key uses a different algorithm byte (0x04) from the log key (0x01), so the two roles can never be confused.

Q. What can a malicious log operator still do?

It cannot delete or rewrite records without breaking consistency proofs, and it cannot show a victim a different history without the honest witnesses refusing, which is scenario 8 of our demo. It can refuse to include a record or go offline; that affects availability, not integrity, and the client then makes a policy decision instead of crashing (NFR-REL-2).

Q. Why Ed25519 rather than RSA or ECDSA?

Ed25519 signatures are deterministic, so there is no per-signature random nonce that can leak the key if the RNG is bad, as happened with ECDSA in the past. Keys are 32 bytes, signatures 64 bytes, verification takes about 60 μ s in our benchmark, and it is standardised in RFC 8032. `verify_strict` additionally rejects non-canonical and small-order signatures.

Q. Why SHA-256 and not BLAKE3?

SHA-256 is FIPS 180-4 standard, it is what RFC 6962 Merkle trees, in-toto and pacman already use, and it is fast enough: hashing is never our bottleneck. Interoperability and auditability matter more here than raw speed.

Q. What is DSSE's PAE, and why not just sign the JSON?

PAE is a length-prefixed encoding: 'DSSEv1', the length and value of the payload type, then the length and value of the body. Signing that binds the type to the content and, thanks to length-prefixing, removes any ambiguity about where one ends and the other begins, so bytes cannot be spliced between messages. Inside the body, the in-toto statement carries its predicate type (rebuild or revocation), which is covered by the signature and checked on decode, so an attestation can never be reinterpreted as a revocation.

Q. What if a rebuilder's private key is stolen?

One compromised rebuilder is below the threshold: Weave needs k independent attestations, and any self-verified rebuilder that reproduces a different digest blocks the install outright (FR-6.4). The key can be revoked with a revocation record that names the leaf by index and digest; revocations stay in history but invalidate the attestation for every client.

Q. How does the client verify inclusion?

It recomputes the path from the leaf hash up to the root using the sibling hashes in the proof and checks that it equals the root in the witness-cosigned checkpoint. Loom goes further: it mirrors every record and recomputes the whole root locally, so the log cannot hide records, and in particular cannot hide revocations.

Numbers every presenter must know



Fact	Value	Where it comes from
Malicious cases blocked (AC-1)	9 / 9 (100%)	loom-eval E1, labelled incident corpus
Benign packages blocked (AC-2)	0 / 7 (0%)	loom-eval E2
Best single mechanism (AC-4)	catches 3 of 9 (sandbox)	loom-eval E4 ablation
Policy decision time	8.3 μ s (target < 500 ms)	loom-eval E5, release build
Ed25519 attestation verification	60 μ s (target < 100 ms)	loom-eval E5, release build
Split views detected (AC-6)	200 / 200	loom-eval E6
Sink hits, same malicious build	0 confined vs 1 unconfined	demo scenario 7
Code base	11 Rust crates, ~12,700 lines, 75 automated tests	cargo test --workspace
Default policy	k = 2 attestations, 2-of-3 witnesses, 72 h quarantine, continuity change = block	SRS Appendix A
Deployment in the demo	mock AUR :7700, Warp :7710, witnesses :7721-3, rebuilders/peers :7731-3, console :7790	demo/run.sh